

Por que a Web funciona?

Fundamentos, System Design e Design de APIs

DIM0547 — Desenvolvimento de Sistemas Web II — Sprint 1 (14/04)

Prof. Fernando · UFRN · 2026.1

Hoje: por que você deveria se importar com o que tem por baixo do seu framework

0 objetivo desta aula

Todo framework web resolve os **mesmos problemas fundamentais**.

Quem conhece os fundamentos:

- Muda de framework (Spring → Chi → Axum → Express) em uma semana
- Debuga em qualquer camada (rede, HTTP, aplicação)
- Projeta APIs que duram 10+ anos

Quem só conhece o framework:

- Fica refém da sintaxe
- Não sabe por que uma coisa funciona
- Escreve APIs que quebram compatibilidade na primeira mudança

Hoje vamos descer até os fundamentos antes de subir para o Chi.

PARTE 1 – Fundamentos da Web

A Web como sistema distribuído

Antes de ser "interface bonita", a Web é um **protocolo** entre máquinas que não se conhecem:

- Centenas de milhões de servidores
- Bilhões de clientes
- **Zero coordenação central**
- **Zero conhecimento prévio** entre as partes
- Funciona há 35+ anos

Como isso é possível?

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Cap. 1). Tese de doutorado que cunhou o termo REST: ics.uci.edu/~fielding/pubs/dissertation/top.htm

Cliente-Servidor: a separação que mudou tudo



Princípio: cliente e servidor **evoluem independentemente**.

- Você troca o backend sem avisar o frontend? Funciona, se o contrato for mantido
- Cliente pode ser browser, mobile, CLI, outro servidor — não importa
- Essa separação é o que permitiu a Web explodir

MDN Web Docs — *Client-server overview*: developer.mozilla.org/en-US/docs/Learn/Server-side/First_steps/Client-Server_overview

HTTP: o protocolo que une tudo

HTTP é **texto puro**. Você pode escrever um cliente HTTP com `telnet`.

```
GET /contacts/42 HTTP/1.1
Host: api.example.com
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 48
```

```
{"id": "42", "name": "Maria", "email": "maria@mail.com"}
```

Por que texto? Para ser legível, debugável, universal. Qualquer linguagem consegue implementar.

RFC 9110 — *HTTP Semantics* (2022, atualização do RFC 7230): <httpwg.org/specs/rfc9110.html>

MDN — *HTTP Messages*: developer.mozilla.org/en-US/docs/Web/HTTP/Messages

Stateless: a propriedade que permite escala

Cada request carrega tudo o que o servidor precisa para respondê-la.

O servidor **não lembra** nada entre requests do mesmo cliente.

Consequência mágica: qualquer servidor pode atender qualquer request.

```
Cliente → [Load Balancer] → Servidor A (request 1)
Cliente → [Load Balancer] → Servidor B (request 2)
Cliente → [Load Balancer] → Servidor C (request 3)
```

Se o servidor **lembrasse** (ex: sessão em memória), o cliente teria que voltar sempre no mesmo servidor. A arquitetura quebra.

Pergunta: onde fica a sessão então? Em banco compartilhado, cookie assinado, JWT... O estado existe — mas **não no servidor da aplicação**.

Fielding — *Statelessness in REST*: ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm#sec_5_1_3

Métodos HTTP: verbos com significado

Método	Semântica	Seguro?	Idempotente?
GET	Ler recurso	✓	✓
POST	Criar / operação não-idempotente	✗	✗
PUT	Substituir recurso completo	✗	✓
PATCH	Atualização parcial	✗	depende
DELETE	Remover recurso	✗	✓
HEAD	Como GET mas sem corpo	✓	✓
OPTIONS	Metadados/capabilities (CORS!)	✓	✓

Seguro = não modifica estado do servidor

Idempotente = N chamadas iguais têm mesmo efeito que 1 chamada

RFC 9110 §9 — *Methods*: httpwg.org/specs/rfc9110.html#methods

MDN — *HTTP request methods*: developer.mozilla.org/en-US/docs/Web/HTTP/Methods

Idempotência: por que isso importa (muito)

Cenário real: cliente mobile faz `POST /orders`. Rede falha. Cliente não recebe resposta.

- **Re-tentar cria pedido duplicado?** Aí está o bug clássico.

Método idempotente = retry seguro.

- `PUT /users/42` com mesmo corpo 10 vezes = 1 vez
- `DELETE /orders/99` 10 vezes = 1 vez (as outras 9 retornam 404 ou 204, tudo bem)
- `POST /orders` 10 vezes = **10 pedidos**

Como fazer POST idempotente? Idempotency-Key header (padrão Stripe).

```
POST /orders HTTP/1.1
Idempotency-Key: 7a9f...
```

Stripe — *Idempotency*: stripe.com/docs/api/idempotent_requests

RFC 9110 §9.2.2 — *Idempotent Methods*: httpwg.org/specs/rfc9110.html#idempotent.methods

Status codes: um contrato compacto

Não é decoração, é **parte do protocolo**.

Família	Significado
1xx	Informacional (raro)
2xx	Sucesso
3xx	Redirecionamento
4xx	Erro do cliente
5xx	Erro do servidor

Os que importam no dia a dia de uma API:

200 OK · 201 Created · 204 No Content · 301 Moved Permanently · 304 Not Modified · 400 Bad Request · 401 Unauthorized · 403 Forbidden · 404 Not Found · 409 Conflict · 422 Unprocessable Entity · 429 Too Many Requests · 500 Internal Server Error · 502 Bad Gateway · 503 Service Unavailable

RFC 9110 §15 — *Status Codes*: <httpwg.org/specs/rfc9110.html#status.codes>

MDN — *HTTP response status codes*: <developer.mozilla.org/en-US/docs/Web/HTTP/Status>

HTTP Cats (para lembrar): <http.cat>

Cache: o recurso mais subutilizado da Web

A Web foi projetada **em cima** de cache. Ignorar cache é jogar fora a principal vantagem do HTTP.

```
Cache-Control: public, max-age=3600
```

```
ETag: "a3f5b9"
```

Fluxo:

1. Cliente pede recurso. Servidor responde com ETag: "a3f5b9"
2. Cliente pede de novo com If-None-Match: "a3f5b9"
3. Servidor responde 304 Not Modified (sem corpo!) — **zero bytes de payload**

Cache funciona em múltiplas camadas:

```
Cliente → Browser cache → CDN → Reverse proxy → Servidor
```

RFC 9111 — *HTTP Caching*: <httpwg.org/specs/rfc9111.html>

MDN — *HTTP caching*: developer.mozilla.org/en-US/docs/Web/HTTP/Caching

CORS: o pesadelo que você precisa entender

Same-Origin Policy: browser **bloqueia** requests JavaScript entre origens diferentes (protocolo + host + porta).

- `https://app.example.com` → `https://api.example.com` é **bloqueado por padrão**

CORS é o mecanismo que diz ao browser: "relaxa, esse servidor aceita".

```
Preflight:  OPTIONS /api/contacts
            Origin: https://app.example.com

Resposta:   Access-Control-Allow-Origin: https://app.example.com
            Access-Control-Allow-Methods: GET, POST, DELETE
            Access-Control-Allow-Headers: Authorization
```

Onde está o erro que todo mundo comete: "CORS é problema do backend". É sim — o backend precisa mandar os headers certos. Mas a **enforcement é no browser**. `curl` ignora CORS.

MDN — *Cross-Origin Resource Sharing (CORS)*: developer.mozilla.org/en-US/docs/Web/HTTP/CORS

Fetch Standard (onde CORS é definido): fetch.spec.whatwg.org/#http-cors-protocol

PARTE 2 – System Design

De 1 servidor para N servidores

Fase 1 (todo projeto acadêmico):

```
Cliente → Servidor → Banco
```

Funciona até ~100 usuários/segundo. Depois trava.

Fase 2 (escalar horizontalmente):

```

      ┌─> Servidor A ─┐
Cliente →┤   Servidor B   ├──> Banco
      └─> Servidor C ─┘
      [Load Balancer]
```

Isso só funciona porque HTTP é **stateless** (parte 1).

Google SRE Book — *The Production Environment at Google*: sre.google/sre-book/production-environment

Load Balancers: mais do que dividir carga

Um load balancer moderno faz:

- **Distribuição** (round-robin, least-connections, hash por IP)
- **Health checks** (tira servidor morto do pool automaticamente)
- **TLS termination** (HTTPS termina no LB, aplicação fala HTTP interno)
- **Rate limiting** (primeira linha de defesa contra DDoS)
- **Sticky sessions** (quando stateless é impossível)

Exemplos reais: nginx, HAProxy, AWS ALB, Cloudflare, Envoy.

NGINX — *What Is Load Balancing?*: nginx.com/resources/glossary/load-balancing

Cloudflare — *What is load balancing?*: cloudflare.com/learning/performance/what-is-load-balancing

Cache em múltiplas camadas

```
Cliente → Browser cache (ms, gratuito)
        → CDN           (10ms, pago)
        → Reverse proxy  (1ms, interno)
        → App cache      (0.1ms, ex: Redis)
        → Banco          (10-100ms)
```

Regra: cada nível evita o próximo.

- 90% dos requests resolvem no browser cache
- 9% dos restantes resolvem na CDN
- 0.9% resolvem no Redis
- 0.1% chegam ao banco

Sem cache, 100% dos requests vão ao banco. Você queima \$\$\$ em IOPS.

Cloudflare — *What is a CDN?*: cloudflare.com/learning/cdn/what-is-a-cdn

AWS — *Caching strategies*: aws.amazon.com/caching

Bancos de dados: além do "tem um Postgres"

Quando o banco é gargalo, as opções clássicas:

- **Replicação** — réplicas de leitura (1 escritor, N leitores)
- **Sharding** — dividir dados por chave (`user_id % N`)
- **Caching** — evitar hit no banco (Redis, Memcached)
- **Read-through / write-through** — cache sincronizado
- **CQRS** — separar escrita e leitura em modelos diferentes

E você percebe que começou com "só um CRUD" e acabou projetando um **sistema distribuído**.

High Scalability blog — arquiteturas reais de empresas: highscalability.com

Designing Data-Intensive Applications (Kleppmann, 2017) — capítulo 5 online gratuito: dataintensive.net/sample.html

O teorema CAP (intuição, não fórmula)

Em um sistema distribuído, quando a rede falha, você escolhe **2 de 3**:

- **C**onsistency — todo nó vê o mesmo dado
- **A**vailability — todo request recebe resposta
- **P**artition tolerance — sistema tolera partições de rede

Em rede real, **P acontece sempre** (cabos caem, datacenters isolam). Você escolhe **C ou A**.

- **CP** (bancos tradicionais): prefiro errar que mentir → retorna erro
- **AP** (DNS, Cassandra): prefiro responder com dado velho a não responder

Dica: a maioria dos sistemas é AP e mente que é CP.

Brewer, E. (2012) — *CAP Twelve Years Later: How the Rules Have Changed*: ieeexplore.ieee.org/document/6133253

Martin Kleppmann — *A Critique of the CAP Theorem*: arxiv.org/abs/1509.05393

Observabilidade: você não debuga o que não vê

Três pilares (vamos voltar em Sprint 4):

- **Logs** — eventos discretos (*"user 42 fez login"*)
- **Métricas** — agregações numéricas (*"p99 latency = 200ms"*)
- **Traces** — fluxo de uma request por múltiplos serviços

Uma request moderna pode passar por 10-15 serviços. Sem traces, você não sabe onde o tempo foi.

"Se você está navegando no escuro, não adianta correr mais."

Google SRE Book — *Monitoring Distributed Systems*: sre.google/sre-book/monitoring-distributed-systems

OpenTelemetry docs: opentelemetry.io/docs

PARTE 3 – Design de APIs

O modelo de maturidade de Richardson

Nível	O que tem	Exemplo
0	HTTP como túnel RPC	POST <code>/api</code> com body <code>{"action":"getUser","id":1}</code>
1	Recursos	POST <code>/users/1</code> (mas só POST)
2	Verbos HTTP + Status	GET <code>/users/1</code> → 200, DELETE <code>/users/1</code> → 204
3	HATEOAS (hipermídia)	Resposta inclui <code>_links</code> para próximas ações

Nível 2 é o que 95% das "REST APIs" modernas realmente são.

Nível 3 é controverso — poderoso em teoria, raríssimo na prática.

Fowler — *Richardson Maturity Model*: martinfowler.com/articles/richardsonMaturityModel.html

Fielding (2008) — *REST APIs must be hypertext-driven*: roy.gbiv.com/untangled/2008/rest-apis-must-be-hypertext-driven

Modelagem de recursos: o núcleo do REST

REST é sobre **recursos** (substantivos), não ações (verbos).

Errado (RPC disfarçado):

```
POST /getUser
POST /createOrder
POST /deleteProduct
```

Certo (REST):

```
GET    /users/42
POST   /orders
DELETE /products/99
```

E quando a operação **não é um CRUD**? "Enviar email de confirmação"?

Opção A: modele como recurso → `POST /confirmation-emails`

Opção B: sub-recurso → `POST /orders/99/confirmation-email`

Opção C: admita RPC para operações específicas — é legítimo

Microsoft — *RESTful web API design*: learn.microsoft.com/en-us/azure/architecture/best-practices/api-design

Google — *API Design Guide*: cloud.google.com/apis/design

Versionamento: a decisão que assombra você em 2 anos

APIs públicas **mudam**. Clientes antigos precisam continuar funcionando.

Três abordagens:

1. **URL:** `/api/v1/contacts` → `/api/v2/contacts`

-  Simples, explícito, cacheável
-  Proliferação de versões

2. **Header:** `Accept: application/vnd.api+json; version=2`

-  URL estável (recurso é o mesmo)
-  Invisível em browser, confuso para debug

3. **Content negotiation:** `Accept: application/vnd.company.v2+json`

-  "HTTP puro"
-  Mais complicado de implementar

Dica honesta: 90% dos casos, use **URL versioning**. Stripe faz. GitHub faz. É o que menos erra.

Stripe — *API versioning*: stripe.com/blog/api-versioning

GitHub — *API versions*: docs.github.com/en/rest/overview/api-versions

Erros: o que retornar quando tudo dá errado

Problema: cada API inventa seu formato de erro.

Solução (que ninguém usa mas é padrão): **RFC 7807 — Problem Details**.

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json

{
  "type": "https://example.com/errors/validation",
  "title": "Validation Error",
  "status": 422,
  "detail": "Email field is invalid",
  "instance": "/contacts/new",
  "errors": [
    {"field": "email", "message": "must be a valid email"}
  ]
}
```

Campos obrigatórios: `type`, `title`, `status`. Extras: livres.

RFC 9457 — *Problem Details for HTTP APIs* (atualização de 7807): datatracker.ietf.org/doc/html/rfc9457

Contratos: OpenAPI é o mínimo civilizado

Uma API sem contrato documentado é uma promessa quebrada esperando para acontecer.

OpenAPI (antes Swagger) é o padrão de fato:

```
paths:
  /contacts/{id}:
    get:
      parameters:
        - name: id
          in: path
          required: true
          schema: { type: string }
      responses:
        '200':
          description: Contato encontrado
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Contact'
        '404':
          description: Não existe
```

Com OpenAPI você ganha: documentação auto-gerada, mocks, testes de contrato, SDKs em N linguagens.

OpenAPI Specification 3.1: spec.openapis.org/oas/v3.1.0

REST não é a única opção

Estilo	Quando faz sentido
REST	APIs públicas, recursos bem definidos, múltiplos clientes
GraphQL	Cliente precisa controlar exatamente o que busca (mobile, agregação)
gRPC	Comunicação interna entre serviços, alta performance, tipado
WebSockets	Real-time bidirecional (chat, colaboração, notifications)
Server-Sent Events	Real-time unidirecional (feeds, dashboards)

Não existe "o melhor". Existe o que serve ao problema.

Vamos ver gRPC e GraphQL no **Sprint 4**. Por enquanto, REST.

Google — *gRPC vs REST*: grpc.io/docs/what-is-grpc/introduction

GraphQL Foundation: graphql.org/learn

Conectando tudo: o que vamos construir

Sua API do Sprint 1 vai ter:

Conceito	Onde aparece
Stateless	Nenhum estado guardado no processo — só em <code>map</code> para testes
Métodos HTTP	GET/POST/PUT/DELETE corretos para cada operação
Status codes	200/201/204/400/404 nos lugares certos
Idempotência	GET, PUT, DELETE naturalmente idempotentes
Validação	<code>validator</code> com <code>422 Unprocessable Entity</code>
Cache	(próximas sprints)
CORS	(quando tiver frontend)
OpenAPI	swaggo na Sprint 1 já
Versionamento	<code>/api/v1/...</code> desde o começo

Tudo isso com Chi, que é uma biblioteca sobre `net/http`.

Sobre a escolha de Chi

Chi **não é um framework**. É uma biblioteca de roteamento que vive em cima do `net/http` padrão.

Você aprende	Porque usa
<code>http.Handler</code> interface	É a interface padrão de Go para HTTP
Middleware = <code>func(http.Handler) http.Handler</code>	Padrão que qualquer lib Go usa
<code>w.WriteHeader</code> , <code>w.Write</code> , <code>r.Body</code>	API da stdlib — nunca vai mudar

Consequência: o que aprender aqui **transfere para qualquer lib Go** (Gin, Echo, Fiber, stdlib puro, Go-kit, Buffalo...).

Gin, Echo e Fiber usam `c.Context` próprio → lock-in.

Spring Boot (Java) usa filosofia **igual** à de Chi: interfaces + composição + DI explícito.

Chi docs: go-chi.io

Alex Edwards — *Let's Go* (livro que usa net/http puro): lets-go.alexedwards.net

Three Dots Labs — *Why Chi is a perfect router for Go*: threedots.tech/post/common-anti-patterns-in-go-web-applications

PARTE 4 – Resolução ao vivo do Ex01

O que o ex01 pede

CRUD de contatos usando o **ServeMux do Go 1.22+** (sem biblioteca externa):

Método	Rota	Retorno
GET	/contacts	200 + array JSON
POST	/contacts	201 + objeto criado
GET	/contacts/{id}	200 ou 404
DELETE	/contacts/{id}	204 ou 404

Conexão com a aula:

- **Stateless** — vamos guardar em `map` só porque é teste
- **Métodos HTTP** — GET/POST/DELETE no padrão Go 1.22 (`"GET /contacts"`)
- **Status codes** — 200, 201, 204, 404 nos lugares certos
- **Content-Type** — `application/json` obrigatório

Agora vamos construir: terminal + editor + TDD.

Ao vivo — siga no seu repositório se quiser acompanhar.

Go 1.22 release notes — *Enhanced ServeMux*: go.dev/blog/routing-enhancements

Go docs — *net/http*: pkg.go.dev/net/http

Encerramento

O que levar para casa hoje:

1. A Web funciona porque tem **contratos simples e universais** (HTTP, status codes, métodos)
2. **Stateless** é o que permite escala horizontal — sem isso, não existiria AWS, Netflix, Google
3. **REST bem feito** = recursos + verbos corretos + status codes corretos + contratos versionados
4. Framework é **ferramenta**, não conhecimento. O que transfere é o fundamento.

Para a semana:

- Ler: RFC 9110 (só o sumário), Fowler — Richardson Maturity Model
- Fazer: Lista 2 (prazo 17/04 sex) e começar Sprint 1

Próxima aula (quinta 16/04): middleware por grupo, validator, OpenAPI. **Chi aplicado ao projeto.**

Referências desta aula

Fundamentos

- [Fielding \(2000\) — REST dissertation](#)
- [MDN — HTTP docs](#)
- [RFC 9110 — HTTP Semantics](#)
- [RFC 9111 — HTTP Caching](#)

System Design

- [Google SRE Book](#)
- [Designing Data-Intensive Applications \(Kleppmann\)](#)
- [High Scalability blog](#)

API Design

- [Richardson Maturity Model \(Fowler\)](#)
- [Google API Design Guide](#)
- [RFC 9457 — Problem Details](#)
- [OpenAPI 3.1](#)

Go e Chi

- [Go 1.22 routing](#)
- [Chi docs](#)
- [Alex Edwards — Let's Go](#)